NOSQL DATABASE DESIGN

LAB SHEETS 2, 3, 4 & 5 — SOLVED PRACTICALS

MongoDB • HBase • Cassandra • Riak KV • Neo4j

Student practical solution book

Commands, sample records, CRUD operations, queries and short explanations.

Note: Replace sample names, IDs and values as required by your teacher. Run commands in the relevant database shell. Some HBase/Riak commands depend on the installed version.

LAB SHEET 2 — NoSQL DATABASE DESIGN

This sheet covers relational vs document databases, MongoDB, Redis-style key-value storage, Cassandra, HBase and Neo4j.

Q1. Create a student database in RDBMS and represent the same data using MongoDB. Compare both approaches.

MySQL stores students in rows and columns. MongoDB stores each student as a BSON document in a collection.

```
-- MySQL
CREATE DATABASE CollegeDB;
USE CollegeDB;
CREATE TABLE Students (
  Student_ID INT PRIMARY KEY,
  Name VARCHAR(50),
  Course VARCHAR(30),
  Semester INT,
  Marks INT
);
INSERT INTO Students VALUES
(1,'Aman','BCA',3,82),
(2,'Riya','BCA',3,91),
(3,'Karan','BCA',4,76),
(4,'Neha','BCA',4,88),
(5,'Rahul','BCA',3,69);

-- MongoDB (mongosh)
use CollegeDB
db.Students.insertMany([
 {Student_ID:1,Name:'Aman',Course:'BCA',Semester:3,Marks:82},
 {Student_ID:2,Name:'Riya',Course:'BCA',Semester:3,Marks:91},
 {Student_ID:3,Name:'Karan',Course:'BCA',Semester:4,Marks:76},
 {Student_ID:4,Name:'Neha',Course:'BCA',Semester:4,Marks:88},
 {Student_ID:5,Name:'Rahul',Course:'BCA',Semester:3,Marks:69}
])
db.Students.find()
```

Comparison: RDBMS uses a fixed schema, tables and relationships; MongoDB uses flexible JSON-like documents. RDBMS is suitable for joins and strict transactions; MongoDB is convenient for nested data and horizontal scaling.

Q2. MongoDB database CollegeDB and Students collection; perform CRUD and count.

Each document contains Student_ID, Name, Course, Semester and Marks.

```
use CollegeDB
db.Students.insertMany([
 {Student_ID:101,Name:'Aman',Course:'BCA',Semester:3,Marks:82},
 {Student_ID:102,Name:'Riya',Course:'BCA',Semester:3,Marks:91},
 {Student_ID:103,Name:'Karan',Course:'BSc',Semester:4,Marks:76},
 {Student_ID:104,Name:'Neha',Course:'BCA',Semester:4,Marks:88},
 {Student_ID:105,Name:'Rahul',Course:'BCom',Semester:2,Marks:69}
])
db.Students.find()
db.Students.find({Course:'BCA'})
db.Students.updateOne({Student_ID:101},{$set:{Marks:90}})
db.Students.deleteOne({Student_ID:105})
db.Students.countDocuments({})
```

Q3. MongoDB Orders collection with embedded customer and products.

Embedding customer and product details keeps the complete order in one document.

```
use CollegeDB
db.Orders.insertMany([
 {_id:1,customer:{id:101,name:'Aman'},products:[{name:'Mouse',qty:2,price:400},{name:'Keyboard',qty:1,price:800}]},
 {_id:2,customer:{id:102,name:'Riya'},products:[{name:'Monitor',qty:1,price:7000}]},
 {_id:3,customer:{id:101,name:'Aman'},products:[{name:'USB',qty:3,price:250}]},
 {_id:4,customer:{id:103,name:'Neha'},products:[{name:'Laptop',qty:1,price:45000}]},
 {_id:5,customer:{id:104,name:'Karan'},products:[{name:'Mouse',qty:1,price:400}]}
])
db.Orders.find({'customer.id':101})
db.Orders.updateOne({_id:1,'products.name':'Mouse'},{$set:{'products.$.qty':3}})
db.Orders.aggregate([
 {$unwind:'$products'},
```

```
 {$group:{_id:'$_id',total:{$sum:{$multiply:['$products.qty','$products.price']}}}}
])
db.Orders.aggregate([
 {$unwind:'$products'},
 {$group:{_id:'$_id',total:{$sum:{$multiply:['$products.qty','$products.price']}}}},
 {$match:{total:{$gt:1000}}}
])
```

Q4. Student information system using a key-value database (Redis).

Use student ID as the key and student name as its value. Redis string commands are SET, GET and DEL.

```
# Redis CLI
SET student:1 "Aman"
SET student:2 "Riya"
SET student:3 "Karan"
SET student:4 "Neha"
SET student:5 "Rahul"

GET student:2
SET student:2 "Riya Sharma"
GET student:2
DEL student:5
KEYS student:*
```

Q5. Cassandra keyspace CollegeDB and Students table.

Cassandra uses CQL. The partition key uniquely identifies each student row.

```
CREATE KEYSPACE CollegeDB
WITH replication = {'class':'SimpleStrategy','replication_factor':1};
USE CollegeDB;

CREATE TABLE Students (
 Student_ID int PRIMARY KEY,
 Name text, Course text, Semester int, Marks int
);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks)
VALUES (1,'Aman','BCA',3,82);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks)
VALUES (2,'Riya','BCA',3,91);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks)
VALUES (3,'Karan','BCA',4,76);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks)
VALUES (4,'Neha','BCA',4,88);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks)
VALUES (5,'Rahul','BCom',2,69);

SELECT * FROM Students;
-- For course lookup, create a query-friendly table:
CREATE TABLE StudentsByCourse (
 Course text, Student_ID int, Name text, Semester int, Marks int,
 PRIMARY KEY (Course, Student_ID)
);
SELECT * FROM StudentsByCourse WHERE Course='BCA';
UPDATE Students SET Marks=95 WHERE Student_ID=2;
DELETE FROM Students WHERE Student_ID=5;
```

Q6. HBase Students table with personal and academic column families.

Column families are defined at table creation. Qualifiers can be added dynamically.

```
# HBase shell
create 'Students', 'personal', 'academic'
put 'Students','101','personal:name','Aman'
put 'Students','101','personal:city','Roorkee'
put 'Students','101','academic:course','BCA'
put 'Students','101','academic:marks','82'
put 'Students','102','personal:name','Riya'
put 'Students','102','academic:course','BCA'
put 'Students','102','academic:marks','91'
get 'Students','101'
scan 'Students'
put 'Students','101','academic:marks','90'
delete 'Students','102','personal:name'
deleteall 'Students','102'
```

Q7. Student–Course graph in Neo4j.

Create Student and Course nodes and connect them using ENROLLED_IN.

```
// Neo4j Browser / Cypher
CREATE (:Student {student_id:1,name:'Aman'});
CREATE (:Student {student_id:2,name:'Riya'});
CREATE (:Student {student_id:3,name:'Karan'});
CREATE (:Student {student_id:4,name:'Neha'});
CREATE (:Student {student_id:5,name:'Rahul'});
CREATE (:Course {course_id:'C1',name:'DBMS'});
CREATE (:Course {course_id:'C2',name:'Python'});
CREATE (:Course {course_id:'C3',name:'Java'});

MATCH (s:Student {student_id:1}),(c:Course {course_id:'C1'})
CREATE (s)-[:ENROLLED_IN]->(c);
MATCH (s:Student {student_id:2}),(c:Course {course_id:'C2'})
CREATE (s)-[:ENROLLED_IN]->(c);
MATCH (s:Student {student_id:3}),(c:Course {course_id:'C1'})
CREATE (s)-[:ENROLLED_IN]->(c);

MATCH (s:Student) RETURN s;
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course {name:'DBMS'}) RETURN s;
MATCH (s:Student {student_id:1})-[:ENROLLED_IN]->(c) RETURN c;
MATCH (s:Student {student_id:4}),(c:Course {course_id:'C3'})
CREATE (s)-[:ENROLLED_IN]->(c);
MATCH (s:Student {student_id:4})-[r:ENROLLED_IN]->(c)
DELETE r;
```

Q8. Compare MongoDB, Cassandra and Neo4j for the same student dataset.

MongoDB is document-oriented, Cassandra is wide-column, and Neo4j is graph-oriented.

Data model: MongoDB document; Cassandra partitioned rows/columns; Neo4j nodes and relationships.
Storage: MongoDB BSON documents; Cassandra SSTables; Neo4j graph records.
Query method: MongoDB query/aggregation, Cassandra CQL, Neo4j Cypher.
Scalability: All can scale horizontally, but Cassandra is designed for distributed high-write workloads; MongoDB for flexible documents; Neo4j for relationship-heavy traversals.

LAB SHEET 3 — MONGODB APPLICATIONS

Run the following examples in mongosh. The examples use a database named Lab3DB.

```
use Lab3DB
```

Q1. Student Activity Tracking System.

Store student details, enrolled courses, attendance, skills and recent activities. Example queries retrieve students by activity conditions.

```
db.students.insertMany([
 {sid:1,name:'Aman',courses:['DBMS','Python'],attendance:85,skills:['SQL','Python'],activities:[{type:'quiz',score:90},{type
 {sid:2,name:'Riya',courses:['DBMS','Java'],attendance:92,skills:['Java','SQL'],activities:[{type:'quiz',score:95}]},
 {sid:3,name:'Karan',courses:['Python'],attendance:65,skills:['Python'],activities:[{type:'lab',score:70}]}
])
db.students.find({attendance:{$gte:75}})
db.students.find({courses:'DBMS'})
db.students.find({'activities.type':'quiz','activities.score':{$gte:90}})
db.students.find({skills:'Python'})
```

Q2. Event Logging System.

Save login/logout and security events. Group failed logins to identify repeated attempts.

```
db.events.insertMany([
 {user:'aman',event:'login',time:new Date(),status:'success'},
 {user:'riya',event:'login',time:new Date(),status:'failed'},
 {user:'riya',event:'login',time:new Date(),status:'failed'},
 {user:'aman',event:'file-upload',time:new Date(),status:'success'},
 {user:'riya',event:'password-change',time:new Date(),status:'success'}
])
db.events.aggregate([
 {$match:{event:'login',status:'failed'}},
 {$group:{_id:'$user',failedAttempts:{$sum:1}}},
 {$match:{failedAttempts:{$gte:2}}}
])
db.events.aggregate([{$group:{_id:'$event',count:{$sum:1}}},{$sort:{count:-1}}])
```

Q3. Mini Blogging Platform.

Each blog document stores author, category, tags, comments and publication information.

```
db.blogs.insertMany([
 {title:'Mongo Basics',author:'Aman',category:'Technology',tags:['mongo','database'],comments:[{by:'Riya',text:'Good'}],publ
 {title:'Java Intro',author:'Riya',category:'Programming',tags:['java','coding'],comments:[],published:true,date:new Date()}
 {title:'NoSQL Notes',author:'Aman',category:'Technology',tags:['nosql','database'],comments:[],published:false,date:new Dat
])
db.blogs.find({tags:'database'})
db.blogs.find({category:'Technology',published:true})
db.blogs.find({tags:{$all:['mongo','database']}})
db.blogs.find({'comments.by':'Riya'})
```

Q4. Flexible Product Catalog.

Different categories can have different attributes in the same collection.

```
db.products.insertMany([
 {name:'Laptop A',category:'laptop',brand:'Dell',ram:16,storage:512,price:55000},
 {name:'Phone B',category:'mobile',brand:'Samsung',camera:50,battery:5000,price:22000},
 {name:'Camera C',category:'camera',brand:'Canon',megapixels:24,lens:'18-55mm',price:40000}
])
db.products.find({category:'laptop',ram:{$gte:8}})
db.products.find({category:'mobile',battery:{$gte:4500}})
db.products.find({category:'camera',megapixels:{$gte:20}})
```

Q5. E-Commerce Order Management System.

Keep customers, products and orders. Use aggregation to calculate a customer's order history and totals.

```
db.orders.insertMany([
 {orderId:1,customerId:101,customer:'Aman',items:[{product:'Mouse',qty:2,price:400},{product:'USB',qty:1,price:250}],status:
 {orderId:2,customerId:102,customer:'Riya',items:[{product:'Laptop',qty:1,price:45000}],status:'shipped'}
])
db.orders.find({customerId:101})
db.orders.aggregate([
```

```
 {$unwind:'$items'},
 {$group:{_id:{customerId:'$customerId',customer:'$customer'},total:{$sum:{$multiply:['$items.qty','$items.price']}}}}
])
```

Q6. Website Analytics System.

Store page visits with user, device, browser, location and timestamp. Aggregation finds popular pages and devices.

```
db.visits.insertMany([
 {page:'/home',user:1,device:'mobile',browser:'Chrome',location:'India',time:new Date()},
 {page:'/about',user:2,device:'desktop',browser:'Edge',location:'India',time:new Date()},
 {page:'/home',user:3,device:'mobile',browser:'Chrome',location:'India',time:new Date()},
 {page:'/products',user:1,device:'mobile',browser:'Chrome',location:'India',time:new Date()}
])
db.visits.aggregate([{$group:{_id:'$page',views:{$sum:1}}},{$sort:{views:-1}}])
db.visits.aggregate([{$group:{_id:'$device',visits:{$sum:1}}},{$sort:{visits:-1}}])
```

Q7. Content Management System.

Store articles with author, category, tags and publication status.

```
db.articles.insertMany([
 {title:'DBMS',author:'Aman',category:'Study',tags:['dbms'],published:true},
 {title:'MongoDB',author:'Riya',category:'Technology',tags:['mongo','nosql'],published:false},
 {title:'Python',author:'Aman',category:'Study',tags:['python'],published:true}
])
db.articles.find({published:true})
db.articles.find({published:false})
db.articles.find({category:'Study',tags:'python'})
db.articles.updateOne({title:'MongoDB'},{$set:{published:true}})
```

Q8. Shopping Cart using embedded documents.

Add, change quantity, remove products and calculate cart value.

```
db.carts.insertOne({userId:101,items:[{productId:1,name:'Mouse',qty:2,price:400},{productId:2,name:'USB',qty:1,price:250}]})
db.carts.updateOne({userId:101},{$push:{items:{productId:3,name:'Keyboard',qty:1,price:800}}})
db.carts.updateOne({userId:101,'items.productId':1},{$set:{'items.$.qty':3}})
db.carts.updateOne({userId:101},{$pull:{items:{productId:2}}})
db.carts.aggregate([{$match:{userId:101}},{$unwind:'$items'},{$group:{_id:'$userId',cartTotal:{$sum:{$multiply:['$items.qty'
```

Q9. Product Review and Rating System.

Store reviews and use aggregation to calculate average rating and review count.

```
db.reviews.insertMany([
 {productId:1,user:'Aman',rating:5,comment:'Excellent'},
 {productId:1,user:'Riya',rating:4,comment:'Good'},
 {productId:2,user:'Karan',rating:3,comment:'Average'}
])
db.reviews.aggregate([
 {$group:{_id:'$productId',averageRating:{$avg:'$rating'},reviewCount:{$sum:1}}}
])
```

Q10. Inventory Monitoring System.

Find products whose stock is at or below their minimum stock level.

```
db.inventory.insertMany([
 {product:'Mouse',stock:5,minStock:10,supplier:'ABC',restockDate:new Date()},
 {product:'Keyboard',stock:30,minStock:10,supplier:'XYZ',restockDate:new Date()},
 {product:'Monitor',stock:3,minStock:5,supplier:'ABC',restockDate:new Date()}
])
db.inventory.find({$expr:{$lte:['$stock','$minStock']}})
db.inventory.updateOne({product:'Mouse'},{$inc:{stock:20}})
```

Q11. Bank Wallet Transfer using MongoDB transaction.

Transactions require a replica set or sharded cluster. Both balance updates commit together or roll back.

```
// Example in mongosh connected to a replica set
const session = db.getMongo().startSession();
const bank = session.getDatabase('Lab3DB');
session.startTransaction();
try {
  bank.wallets.updateOne({user:'Aman',balance:{$gte:500}},{$inc:{balance:-500}});
  bank.wallets.updateOne({user:'Riya'},{$inc:{balance:500}});
```

```
  session.commitTransaction();
} catch (e) {
  session.abortTransaction();
  throw e;
} finally {
  session.endSession();
}
// Create sample wallets first:
db.wallets.insertMany([{user:'Aman',balance:2000},{user:'Riya',balance:1000}])
```

Q12. User Notification System.

Retrieve unread notifications and mark a notification as read.

```
db.notifications.insertMany([
 {userId:1,message:'Order placed',type:'order',read:false,createdAt:new Date()},
 {userId:1,message:'Welcome',type:'system',read:true,createdAt:new Date()},
 {userId:2,message:'New message',type:'message',read:false,createdAt:new Date()}
])
db.notifications.find({userId:1,read:false})
db.notifications.updateMany({userId:1,read:false},{$set:{read:true}})
db.notifications.find({userId:1})
```

Q13. Course Learning Platform.

Store course, instructor, modules and student progress. Query students by progress.

```
db.learning.insertMany([
 {student:'Aman',course:'MongoDB',instructor:'Mr X',modules:[{name:'Intro',done:true},{name:'CRUD',done:true}],progress:100,
 {student:'Riya',course:'MongoDB',instructor:'Mr X',modules:[{name:'Intro',done:true},{name:'CRUD',done:false}],progress:50,
 {student:'Karan',course:'Python',instructor:'Ms Y',modules:[{name:'Basics',done:true}],progress:80,completed:false}
])
db.learning.find({course:'MongoDB',progress:{$gte:50}})
db.learning.find({completed:true})
db.learning.find({progress:{$lt:100}})
```

Q14. Real-Time Order Tracking System.

Keep current status and a history array of status changes.

```
db.tracking.insertOne({orderId:501,status:'Placed',history:[{status:'Placed',at:new Date()}]})
db.tracking.updateOne({orderId:501},{$set:{status:'Confirmed'},$push:{history:{status:'Confirmed',at:new Date()}}})
db.tracking.updateOne({orderId:501},{$set:{status:'Shipped'},$push:{history:{status:'Shipped',at:new Date()}}})
db.tracking.updateOne({orderId:501},{$set:{status:'Delivered'},$push:{history:{status:'Delivered',at:new Date()}}})
db.tracking.findOne({orderId:501})
```

Q15. Flexible News Database.

News articles can have optional fields such as author, location, category, tags, images and sources.

```
db.news.insertMany([
 {title:'Tech Update',author:'Aman',location:'India',category:'Technology',tags:['AI','tech'],image:'ai.jpg',source:'News A'
 {title:'City News',location:'Roorkee',category:'Local',tags:['city'],source:'News B'},
 {title:'Sports Update',author:'Riya',category:'Sports',tags:['cricket'],source:'News C'}
])
db.news.find({category:'Technology'})
db.news.find({location:'Roorkee'})
db.news.find({tags:'cricket'})
db.news.find({author:{$exists:true}})
```

LAB SHEET 4 — HBASE AND CASSANDRA

Part A — Apache HBase. Start HBase and open its shell using `hbase shell`.

Q1. Install/configure HBase; create Student with personal and academic families; insert five records.

Run these commands after HBase is running.

```
create 'Student', 'personal', 'academic'
put 'Student','101','personal:name','Aman'
put 'Student','101','personal:city','Roorkee'
put 'Student','101','academic:course','BCA'
put 'Student','101','academic:semester','3'
put 'Student','102','personal:name','Riya'
put 'Student','102','academic:course','BCA'
put 'Student','103','personal:name','Karan'
put 'Student','103','academic:course','BCA'
put 'Student','104','personal:name','Neha'
put 'Student','104','academic:course','BCA'
put 'Student','105','personal:name','Rahul'
put 'Student','105','academic:course','BCA'
scan 'Student'
```

Q2. HBase CRUD on Employee table.

HBase shell equivalents: create table, put, get/scan and delete.

```
create 'Employee', 'info', 'job'
put 'Employee','E1','info:name','Aman'
put 'Employee','E1','info:city','Roorkee'
put 'Employee','E1','job:role','Developer'
put 'Employee','E2','info:name','Riya'
get 'Employee','E1'
scan 'Employee'
put 'Employee','E1','job:role','Senior Developer'
delete 'Employee','E2','info:name'
deleteall 'Employee','E2'
```

Q3. Event Logging table.

Use event_id as row key; store timestamp, user, event type and status in families.

```
create 'EventLog', 'event', 'user'
put 'EventLog','EV1','event:timestamp','2026-11-02T10:00:00'
put 'EventLog','EV1','event:type','login'
put 'EventLog','EV1','event:status','success'
put 'EventLog','EV1','user:user_id','U101'
put 'EventLog','EV2','event:timestamp','2026-11-02T10:05:00'
put 'EventLog','EV2','event:type','login'
put 'EventLog','EV2','event:status','failed'
put 'EventLog','EV2','user:user_id','U102'
get 'EventLog','EV1'
scan 'EventLog'
```

Q4. HBase Content Management System.

Separate content, author and metadata into column families.

```
create 'CMS', 'content', 'author', 'metadata'
put 'CMS','A1','content:title','NoSQL Introduction'
put 'CMS','A1','content:body','Introduction to NoSQL databases'
put 'CMS','A1','author:name','Aman'
put 'CMS','A1','metadata:status','draft'
put 'CMS','A1','metadata:category','Technology'
put 'CMS','A1','metadata:status','published'
get 'CMS','A1'
delete 'CMS','A1','content:body'
get 'CMS','A1'
```

Q5. Blogging Platform in HBase.

Use separate families for blog, author and comments. Each blog ID is a row key.

```
create 'Blogs', 'blog', 'author', 'comments'
```

```
put 'Blogs','B1','blog:title','MongoDB Basics'
put 'Blogs','B1','blog:category','Database'
put 'Blogs','B1','author:name','Aman'
put 'Blogs','B1','comments:C1','Very useful'
put 'Blogs','B2','blog:title','Java Basics'
put 'Blogs','B2','author:name','Riya'
put 'Blogs','B3','blog:title','Python Basics'
put 'Blogs','B3','author:name','Karan'
get 'Blogs','B1'
scan 'Blogs'
```

Q6. Demonstrate column-family model and qualifiers.

Column families are fixed at table creation; qualifiers such as name, city, course and marks are flexible.

```
create 'StudentCF', 'personal', 'academic'
put 'StudentCF','1','personal:name','Aman'
put 'StudentCF','1','personal:city','Roorkee'
put 'StudentCF','1','academic:course','BCA'
put 'StudentCF','1','academic:marks','82'
describe 'StudentCF'
get 'StudentCF','1'
scan 'StudentCF'
```

Q7. WebsiteCounter page-view counters.

Increment a counter atomically with `incr`; read its value with `get_counter`.

```
create 'WebsiteCounter', 'counter'
incr 'WebsiteCounter','home','counter:views',1
incr 'WebsiteCounter','home','counter:views',1
incr 'WebsiteCounter','about','counter:views',1
get_counter 'WebsiteCounter','home','counter:views'
get 'WebsiteCounter','home'
```

Q8. Session data with TTL.

Set TTL in seconds for a column family. A value of 3600 means one hour.

```
create 'Sessions', {NAME=>'session', TTL=>3600}
put 'Sessions','S1','session:user_id','U101'
put 'Sessions','S1','session:status','active'
get 'Sessions','S1'
# After the TTL expires, the session cell is eligible for removal.
```

Q9. Row-key design for horizontal scaling.

Use a distributed prefix to avoid monotonically increasing keys writing to one region. Example: hash bucket + user ID.

```
create 'LargeApp', 'data'
put 'LargeApp','00#USER101','data:name','Aman'
put 'LargeApp','01#USER102','data:name','Riya'
put 'LargeApp','02#USER103','data:name','Karan'
put 'LargeApp','03#USER104','data:name','Neha'
scan 'LargeApp'
# Pre-split example at table creation:
create 'PreSplit', 'd', {SPLITS=>['1','2','3']}
```

Q10. HBase regions and RegionServers.

A table is divided into regions by row-key ranges. Regions are assigned to RegionServers; splitting distributes load.

```
# Example table with split points
create 'RegionalData', 'info', {SPLITS=>['m','t']}
put 'RegionalData','alice','info:city','Delhi'
put 'RegionalData','naman','info:city','Roorkee'
put 'RegionalData','zoya','info:city','Mumbai'
scan 'RegionalData'
# Admin-level region/cluster inspection is commonly done through HBase UI
# or HBase Master/RegionServer administration tools.
```

Part B — Apache Cassandra. Open `cqlsh` after starting Cassandra.

Q11. Install/configure Cassandra; create CollegeDB keyspace and Students table.

Replication factor 1 is suitable for a single-node classroom setup.

```
CREATE KEYSPACE CollegeDB
WITH replication = {'class':'SimpleStrategy','replication_factor':1};
USE CollegeDB;
CREATE TABLE Students (
 student_id int PRIMARY KEY,
 name text, course text, semester int, marks int
);
INSERT INTO Students VALUES (1,'Aman','BCA',3,82);
INSERT INTO Students VALUES (2,'Riya','BCA',3,91);
INSERT INTO Students VALUES (3,'Karan','BCA',4,76);
INSERT INTO Students VALUES (4,'Neha','BCA',4,88);
INSERT INTO Students VALUES (5,'Rahul','BCA',3,69);
SELECT * FROM Students;
```

Q12. Cassandra CRUD on Employees table.

Cassandra updates/deletes require the primary key in the WHERE clause.

```
USE CollegeDB;
CREATE TABLE Employees (
 emp_id int PRIMARY KEY, name text, department text, salary int
);
INSERT INTO Employees VALUES (1,'Aman','IT',30000);
INSERT INTO Employees VALUES (2,'Riya','HR',28000);
SELECT * FROM Employees;
UPDATE Employees SET salary=35000 WHERE emp_id=1;
DELETE FROM Employees WHERE emp_id=2;
SELECT * FROM Employees;
```

Q13. Cassandra event logging application.

Design the primary key for queries by user and time. The clustering column sorts events by time.

```
CREATE TABLE EventsByUser (
 user_id text, event_time timestamp, event_type text, status text,
 PRIMARY KEY (user_id, event_time)
) WITH CLUSTERING ORDER BY (event_time DESC);
INSERT INTO EventsByUser (user_id,event_time,event_type,status)
VALUES ('U1','2026-11-02 10:00:00','login','success');
INSERT INTO EventsByUser (user_id,event_time,event_type,status)
VALUES ('U1','2026-11-02 10:05:00','upload','success');
SELECT * FROM EventsByUser
WHERE user_id='U1'
AND event_time >= '2026-11-02 00:00:00'
AND event_time < '2026-11-03 00:00:00';
```

Q14. Cassandra availability and consistency.

Create a replicated keyspace. Use consistency levels ONE, QUORUM and ALL for reads/writes. In a single-node setup with RF=1, all three can succeed if the node is available.

```
CREATE KEYSPACE Shop
WITH replication = {'class':'SimpleStrategy','replication_factor':1};
USE Shop;
CREATE TABLE stock (product_id int PRIMARY KEY, quantity int);
CONSISTENCY ONE;
INSERT INTO stock (product_id,quantity) VALUES (1,20);
CONSISTENCY QUORUM;
SELECT * FROM stock;
CONSISTENCY ALL;
SELECT * FROM stock;
-- In a multi-node cluster, QUORUM requires a majority of replicas;
-- ALL requires every replica to respond.
```

Q15. Column-oriented NoSQL application using HBase or Cassandra.

Example: event logging using Cassandra. It demonstrates table creation, insertion, querying, updating and deletion. Cassandra provides tunable consistency, replication and horizontal scaling.

```
CREATE KEYSPACE EventApp
WITH replication = {'class':'SimpleStrategy','replication_factor':1};
USE EventApp;
CREATE TABLE EventLog (
 event_id uuid PRIMARY KEY, user_id text, event_type text,
 event_time timestamp, status text
);
INSERT INTO EventLog (event_id,user_id,event_type,event_time,status)
```

```
VALUES (uuid(),'U1','login',toTimestamp(now()),'success');
SELECT * FROM EventLog;
-- Use the event_id returned by SELECT in the following commands:
UPDATE EventLog SET status='reviewed'
WHERE event_id=11111111-1111-1111-1111-111111111111;
DELETE FROM EventLog
WHERE event_id=11111111-1111-1111-1111-111111111111;
```

The UUID shown in the final UPDATE/DELETE is a placeholder. Replace it with an actual event_id returned by SELECT.

LAB SHEET 5 — RIAK KV AND NEO4J

Part A — Riak KV. Riak KV versions differ: the examples below use the common HTTP API (curl) and JSON values. Ensure Riak is running and its HTTP port is reachable.

Q1. Install/configure Riak KV; create students bucket and perform key-value CRUD.

In Riak, buckets group objects. Store JSON under a key, retrieve it, update it by PUT and delete it.

```
# Create/store student records (Riak HTTP API)
curl -X PUT http://localhost:8098/buckets/students/keys/101 \
 -H 'Content-Type: application/json' -d '{"id":101,"name":"Aman","course":"BCA"}'
curl -X PUT http://localhost:8098/buckets/students/keys/102 \
 -H 'Content-Type: application/json' -d '{"id":102,"name":"Riya","course":"BCA"}'
curl http://localhost:8098/buckets/students/keys/101
curl -X PUT http://localhost:8098/buckets/students/keys/101 \
 -H 'Content-Type: application/json' -d '{"id":101,"name":"Aman Saini","course":"BCA"}'
curl -X DELETE http://localhost:8098/buckets/students/keys/102
```

Q2. Riak user_sessions bucket.

Use session ID as key and store user ID, login time and status in JSON.

```
curl -X PUT http://localhost:8098/buckets/user_sessions/keys/S001 \
 -H 'Content-Type: application/json' \
 -d '{"session_id":"S001","user_id":"U101","login_time":"2026-11-02T10:00:00","status":"active"}'
curl http://localhost:8098/buckets/user_sessions/keys/S001
```

Q3. Riak user profiles and preferences.

Store profile and preferences as a JSON object; demonstrate PUT and GET.

```
curl -X PUT http://localhost:8098/buckets/profiles/keys/U101 \
 -H 'Content-Type: application/json' \
 -d '{"name":"Aman","email":"aman@example.com","city":"Roorkee","language":"Hindi","theme":"light","notifications":true}'
curl http://localhost:8098/buckets/profiles/keys/U101
```

Q4. Shopping cart using Riak KV.

Store one cart object per user. Application code reads the JSON, changes the items and writes it back.

```
curl -X PUT http://localhost:8098/buckets/carts/keys/U101 \
 -H 'Content-Type: application/json' \
 -d '{"user":"U101","items":[{"product":"Mouse","qty":2,"price":400},{"product":"USB","qty":1,"price":250}]}'
curl http://localhost:8098/buckets/carts/keys/U101
# Update the full JSON value to change quantity or remove a product.
curl -X PUT http://localhost:8098/buckets/carts/keys/U101 \
 -H 'Content-Type: application/json' \
 -d '{"user":"U101","items":[{"product":"Mouse","qty":3,"price":400}]}'
```

Q5. Demonstrate data structures in Riak KV.

Riak KV stores opaque values; JSON strings, arrays and objects can be stored. The application interprets the data structure.

```
curl -X PUT http://localhost:8098/buckets/demo/keys/string1 -H 'Content-Type: text/plain' -d 'Hello'
curl -X PUT http://localhost:8098/buckets/demo/keys/json1 -H 'Content-Type: application/json' -d '{"name":"Aman","age":21}'
curl -X PUT http://localhost:8098/buckets/demo/keys/list1 -H 'Content-Type: application/json' -d '["SQL","Python","Java"]'
curl http://localhost:8098/buckets/demo/keys/string1
curl http://localhost:8098/buckets/demo/keys/json1
curl http://localhost:8098/buckets/demo/keys/list1
```

Q6. Consistency in Riak KV.

Riak uses quorum parameters. For example, `r=quorum` and `w=quorum` request quorum reads and writes. Exact values depend on bucket properties and cluster size.

```
# Quorum-style read and write
curl -X PUT 'http://localhost:8098/buckets/students/keys/103?w=quorum' \
 -H 'Content-Type: application/json' -d '{"id":103,"name":"Karan"}'
curl 'http://localhost:8098/buckets/students/keys/103?r=quorum'
# In a cluster, quorum improves consistency but may reduce availability
# if too many replicas are unreachable.
```

Q7. Riak scaling with multiple buckets.

Create multiple buckets and add records. Riak distributes objects across nodes using consistent hashing and the ring.

```
curl -X PUT http://localhost:8098/buckets/students/keys/201 -H 'Content-Type: application/json' -d '{"name":"Neha"}'
curl -X PUT http://localhost:8098/buckets/products/keys/P1 -H 'Content-Type: application/json' -d '{"name":"Mouse","price":4
curl -X PUT http://localhost:8098/buckets/orders/keys/O1 -H 'Content-Type: application/json' -d '{"user":"U101","total":800}
curl http://localhost:8098/buckets/products/keys/P1
# Add nodes to a cluster using the Riak cluster-management procedure
# for your installed Riak release.
```

Q8. Multi-operation shopping-cart scenario.

Riak KV does not provide general multi-key ACID transactions. Keep related cart data in one object or coordinate updates in application logic.

```
# Read the cart, update its JSON in the application, then PUT the complete object.
curl http://localhost:8098/buckets/carts/keys/U101
curl -X PUT http://localhost:8098/buckets/carts/keys/U101 \
 -H 'Content-Type: application/json' \
 -d '{"user":"U101","items":[{"product":"Mouse","qty":3,"price":400},{"product":"Keyboard","qty":1,"price":800}],"total":200
# The application calculates total = sum(qty * price).
```

Q9. Query-by-data for Riak user records.

Riak KV is primarily key-based. Query-by-data requires secondary indexes, Riak Search (if enabled), or application-side filtering. A JSON document alone does not automatically make arbitrary fields searchable.

```
# Store a record with indexed fields (index support depends on bucket type/version)
curl -X PUT http://localhost:8098/buckets/profiles/keys/U102  -H 'Content-Type: application/json'  -H 'X-Riak-Index-city_bir
# Secondary-index query (2i), where supported:
curl 'http://localhost:8098/buckets/profiles/index/city_bin/Roorkee'
```

Q10. Set operations in Riak.

Use Riak sets (CRDT data types) through the supported datatype API/client. Conceptually add/remove elements and read the set; exact endpoint syntax varies by Riak version/client.

```
# Conceptual set operations using a Riak client datatype API:
# create/update set key "interests:U101"
# add("music"), add("coding"), add("cricket")
# remove("music")
# contains("coding") -> true
# intersection is calculated by reading two sets and intersecting
# their elements in application code.
# Note: Riak Set CRDTs require bucket types configured for datatype use.
```

Part B — Neo4j. Run the following Cypher in Neo4j Browser. Each task uses nodes and relationships; use MERGE to avoid duplicate nodes.

Q11. Neo4j graph with Student, Course and Department nodes.

Create nodes and relationships, then visualize the graph.

```
CREATE (s1:Student {id:1,name:'Aman'}),
       (s2:Student {id:2,name:'Riya'}),
       (c1:Course {id:'C1',name:'BCA'}),
       (d1:Department {id:'D1',name:'Computer Applications'});
MATCH (s:Student {id:1}),(c:Course {id:'C1'})
MERGE (s)-[:ENROLLED_IN]->(c);
MATCH (c:Course {id:'C1'}),(d:Department {id:'D1'})
MERGE (c)-[:BELONGS_TO]->(d);
MATCH (s)-[r]->(n) RETURN s,r,n;
```

Q12. Social network with Person nodes and FRIENDS_WITH, FOLLOWS, WORKS_AT.

Create people and relationships; query friends and followers.

```
CREATE (a:Person {name:'Aman'}),
       (b:Person {name:'Riya'}),
       (c:Person {name:'Karan'}),
       (co:Company {name:'ABC Ltd'});
MATCH (a:Person {name:'Aman'}),(b:Person {name:'Riya'})
MERGE (a)-[:FRIENDS_WITH]->(b);
MATCH (c:Person {name:'Karan'}),(a:Person {name:'Aman'})
```

```
MERGE (c)-[:FOLLOWS]->(a);
MATCH (a:Person {name:'Aman'}),(co:Company {name:'ABC Ltd'})
MERGE (a)-[:WORKS_AT]->(co);
MATCH (:Person {name:'Aman'})-[:FRIENDS_WITH]-(friend:Person)
RETURN friend;
MATCH (follower:Person)-[:FOLLOWS]->(:Person {name:'Aman'})
RETURN follower;
```

Q13. Shopping/e-commerce graph.

Connect Customer, Product, Category and Order nodes.

```
CREATE (c:Customer {id:1,name:'Aman'}),
       (p:Product {id:10,name:'Mouse'}),
       (cat:Category {name:'Accessories'}),
       (o:Order {id:501,status:'Placed'});
MATCH (c:Customer {id:1}),(o:Order {id:501})
MERGE (c)-[:PLACED]->(o);
MATCH (o:Order {id:501}),(p:Product {id:10})
MERGE (o)-[:CONTAINS {quantity:2}]->(p);
MATCH (p:Product {id:10}),(cat:Category {name:'Accessories'})
MERGE (p)-[:IN_CATEGORY]->(cat);
MATCH (:Customer {id:1})-[:PLACED]->(:Order)-[:CONTAINS]->(p:Product)
RETURN p;
```

Q14. Transactions, consistency and availability in Neo4j.

A transaction groups writes. If an error occurs before commit, the transaction is rolled back. In Neo4j Browser, run the statements in one transaction using the driver's transaction API for application work.

```
// Cypher statements to run inside one driver-managed transaction:
CREATE (c:Customer {id:99,name:'Test Customer'});
MATCH (c:Customer {id:99})
SET c.status='active';
MATCH (c:Customer {id:99})
RETURN c;
// Commit when all statements succeed; otherwise rollback in the driver.
// Example Cypher transaction block support depends on Neo4j version.
```

Q15. Small graph database application with CRUD and Cypher queries.

Example student-course application. Graph databases efficiently answer relationship queries such as enrolled courses and classmates.

```
// CREATE
MERGE (s:Student {id:201})
SET s.name='Aman', s.semester=3;
MERGE (c:Course {id:'DB1'})
SET c.name='DBMS';
MATCH (s:Student {id:201}),(c:Course {id:'DB1'})
MERGE (s)-[:ENROLLED_IN]->(c);

// READ
MATCH (s:Student {id:201})-[:ENROLLED_IN]->(c:Course)
RETURN s.name,c.name;

// UPDATE
MATCH (s:Student {id:201})
SET s.semester=4;

// DELETE relationship, then optionally delete node
MATCH (s:Student {id:201})-[r:ENROLLED_IN]->(c:Course)
DELETE r;
MATCH (s:Student {id:201})
DETACH DELETE s;
```

QUICK REVISION — COMMAND SUMMARY

MongoDB: insertOne/insertMany, find/findOne, updateOne/updateMany, deleteOne, aggregate.
HBase: create, put, get, scan, delete, deleteall, incr.
Cassandra: CREATE KEYSPACE, CREATE TABLE, INSERT, SELECT, UPDATE, DELETE, CONSISTENCY.
Riak KV: PUT/GET/DELETE objects by bucket and key; quorum parameters and secondary indexes depend on configuration.
Neo4j: CREATE/MERGE nodes and relationships; MATCH to query; SET to update; DELETE/DETACH DELETE to remove.

Important practical note: Cassandra does not support arbitrary filtering efficiently without suitable keys/indexes; HBase scans can be expensive; Riak KV is key-value first and not a relational query engine; Neo4j is best when relationships are central.